

CareerConnect - AI Usage Log

Name: Yousef Yousef

Sprint: 1

Entry 1

Entry number: 1

Date: 2026-09-22

AI tool/model: Claude (Anthropic), claude-opus-5-5

Task ID/Title: Sprint 1 – Defining my role and responsibilities

Purpose of AI Use: Planning

Chat Link or Prompt/Response: See Appendix, Entry 1

AI-Suggested Content: A summary of the Sprint 1 requirements, a proposed backend split between Catalin and me.

Validation: Compared the split against our team wiki roles and discussed it at the team meeting, where the split was agreed.

Decision: Modified before use. I corrected it to remove SQA and security from my scope, since those belong to other members.

Reflection: Helpful for clarifying my scope before the meeting, but it needed correcting against our team's actual roles.

Responsible Person: Yousef Yousef

Entry 2

Entry number: 2

Date: 2026-09-24

AI tool/model: Claude (Anthropic), claude-opus-5-5

Task ID/Title: #4 – Help setting up databases (Supabase project, profiles and resumes tables, resumes storage bucket)

Purpose of AI Use: Learning Supabase setup with guidance

Chat Link or Prompt/Response: See Appendix, Entry 2

AI-Suggested Content: Supabase setup steps, storage bucket settings, and SQL to record the bucket settings in the repo.

Validation: I built everything myself in the Supabase dashboard. I tested profiles by inserting a test user and checking the defaults, confirmed cascade delete by deleting that user, and checked the table Definitions for the not-null and cascade settings.

Decision: Modified before use. I linked resumes to profiles instead of auth.users, and made file_name required.

Reflection: Useful as a guide for learning Supabase. I understand and tested what I built and got the hang of it.

Responsible Person: Yousef Yousef

Entry 3

Entry number: 3

Date: 2026-09-24

AI tool/model: Claude (Anthropic), claude-opus-5-5

Task ID/Title: Git workflow for committing the database schema

Purpose of AI Use: Learning branches and pull requests

Chat Link or Prompt/Response: See Appendix, Entry 3

AI-Suggested Content: How to create feature branches from dev, commit files, and open PRs into dev.

Validation: Followed our team's agreed workflow. Created feature/database-profiles and feature/database-resumes from dev and opened a PR for each into dev.

Decision: Accepted.

Reflection: Helped me understand the PR process and follow the team workflow.

Responsible Person: Yousef Yousef

Entry 4

Entry number: 4

Date: 2026-09-26 (follow-up 2026-09-27)

AI tool/model: Claude (Anthropic), claude-opus-5-5

Task ID/Title: PR #46 review - Resume upload frontend (User Story #3, #19)

Purpose of AI Use: Code review and debugging. Checking a teammate's pull request, finding how to verify it locally, and understanding the lint errors it produced.

Chat Link or Prompt/Response: Full prompts and responses in Appendix 4.

AI-Suggested Content: The AI read PR #46 from the repository and reported that the page only does frontend validation (PDF type, 5 MB limit) and does not upload anything to Supabase, even though it shows "File uploaded successfully". It flagged that the component name was lowercase (`uploadResumePage`), which breaks React's rule that components start with a capital letter, and predicted the lint rule for hooks would flag it. It gave the commands to check out the PR locally and test it (`tsc`, `lint`, `dev server`) plus a list of manual test cases, and drafted a review comment. When I asked why not keep `camelCase`, it explained that React components use `PascalCase` while normal functions use `camelCase`, and that every other component in our repo already follows this.

Validation: I checked out PR #46 locally and ran the checks myself. `"npx eslint src/app/upload-resume/page.tsx"` returned 3 errors (`react-hooks/rules-of-hooks` on lines 6, 7 and 8), confirming the naming issue. I ran the app and tested the page in the browser: a PDF over 5 MB was correctly rejected with "File size exceeds 5 MB limit". I confirmed nothing appeared in Supabase Storage after clicking Upload, confirming the upload was not yet connected. I compared the component name against the other pages in the repo (`LoginPage`, `SignupPage`, `DashboardPage`), which all use `PascalCase`.

Decision: Accepted. I fixed the component name (`uploadResumePage` to `UploadResumePage`) myself in my dashboard-logout PR (#56). The fake success message was resolved later when Catalin connected the upload to Supabase in PR #57, which I tested and merged.

Reflection: The AI caught an issue that did not show up when running the app with `npm run dev`, since the dev server does not run lint or type checks. I learned the difference between `tsc` (type checking), ESLint (code rules and conventions) and the dev server, and why React needs capitalized component names. Running lint before approving a PR is now part of how I review code.

Responsible Person: Yousef Yousef

Appendix

Prompts and responses are copied from the original chats.

Entry number: 1

Date: 2026-09-22

AI tool/model: Claude (Anthropic), claude-opus-5-5

Task ID/Title: Sprint 1 – Defining my role and responsibilities

Purpose of AI Use: Planning

Chat Link or Prompt/Response: See Appendix, Entry 1

AI-Suggested Content: A summary of the Sprint 1 requirements, a proposed backend split between Catalin and me.

Validation: Compared the split against our team wiki roles and discussed it at the team meeting, where the split was agreed.

Decision: Modified before use. I corrected it to remove SQA and security from my scope, since those belong to other members.

Reflection: Helpful for clarifying my scope before the meeting, but it needed correcting against our team's actual roles.

Responsible Person: Yousef Yousef

=====

APPENDIX 1 - Defining my role and responsibilities (full prompts and responses)

=====

--- Prompt 1 (2026-09-22) ---

tell me what you know about sprint 1, as well as current progress from github context [course outline, Sprint 1 document, README and AI_Log instructions attached]

--- Response 1 ---

Sprint 1 is worth 3% of the course, graded out of 15. Setup instructions are empty. Proposed features only lists 'Skill gap map.' There's no Definition of Ready, no Definition of Done, and no concrete code review rule. AI provider is still TBD.

--- Prompt 2 (2026-09-22) ---

[pasted team wiki: roles and proposed features] above ive pasted the wiki, ive taken responsibility of sharing backend dev and the ai components. explain my role thoroughly, and how it connects with the overall project

--- Response 2 ---

You sit on two of the four teams in the wiki: Backend (with Catalin) and SQA + AI (with Daksikan). What the AI does: skill extraction. What regular code does: the gap calculation. Once skills are stored, the ranking in the wiki example needs no AI.

--- Prompt 3 (2026-09-22) ---

Okay, listen, I'm not doing SQA. I said I'll take backend with Catalin, and I want to take on the AI features. So SQA is not my role. So help me decide from now what to take from back end as well as what AI components I'm doing

--- Response 3 ---

Got it: Your role is backend (shared with Catalin) + all AI features. Take the part of the backend that feeds the AI. You (Yousef): User profiles API | Resume upload and storage | Skills data | All AI endpoints. Catalin: Authentication | Roles and access control | Job postings API | Job search and filtering | Applications API. Shared: database schema design (Sprint 1)

--- Prompt 4 (2026-09-22) ---

So, okay, this is the split, right? I would do user profiles API. I want you to break all of these down in the proposed split for my section, Yousef

--- Response 4 ---

Here's what each item in your column actually involves: the tables you'd create, the endpoints you'd build, the rules you'd enforce, and what 'done' looks like. 1. User Profiles API. 2. Resume upload and storage. 3. Skills data. 4. AI endpoints. 5. AI provider selection

Entry number: 2

Date: 2026-09-24

AI tool/model: Claude (Anthropic), claude-opus-5-5

Task ID/Title: #4 – Help setting up databases (Supabase project, profiles and resumes tables, resumes storage bucket)

Purpose of AI Use: Learning Supabase setup with guidance

Chat Link or Prompt/Response: See Appendix, Entry 2

AI-Suggested Content: Supabase setup steps, storage bucket settings, and SQL to record the bucket settings in the repo.

Validation: I built everything myself in the Supabase dashboard. I tested profiles by inserting a test user and checking the defaults, confirmed cascade delete by deleting that user, and checked the table Definitions for the not-null and cascade settings.

Decision: Modified before use. I linked resumes to profiles instead of auth.users, and made file_name required.

Reflection: Useful as a guide for learning Supabase. I understand and tested what I built and got the hang of it.

Responsible Person: Yousef Yousef

=====

APPENDIX 2 - #4 Help setting up databases (full prompts and responses)

=====

--- Prompt 1 (2026-09-24) ---

[Screenshots of the Supabase organization and project creation forms, including security settings.]

--- Response 1 ---

Educational is the right choice. For Plan, choose Free (\$0/month). Enable Data API: keep it checked.
Enable automatic RLS: check this one.

--- Prompt 2 (2026-09-24) ---

does it matter the order? also how do i commit this

--- Response 2 ---

The column order doesn't matter to the database, id comes in as type int8. Change it to uuid and clear its default value. On delete: Cascade. You copy the table's definition into a file

--- Prompt 3 (2026-09-24) ---

does this look right? and are default values meant to be null [screenshot of columns]

--- Response 3 ---

role: in its Default value box, type job_seeker. NULL just means 'empty unless something fills it.' That's correct for id: full_name

--- Prompt 4 (2026-09-24) ---

but what about the resume storage files, how do i add that to the pr too

--- Response 4 ---

Option 1: just describe it (easiest). Option 2: record it in the file, like the tables. Add this at the bottom of supabase/resumes.sql: insert into storage.buckets (id, name, public, file_size_limit, allowed_mime_types) values ('resumes', 'resumes', false, 5242880, array['application/pdf']); Don't run it in Supabase, because the bucket already exists.

Entry number: 3

Date: 2026-09-24

AI tool/model: Claude (Anthropic), claude-opus-5-5

Task ID/Title: Git workflow for committing the database schema

Purpose of AI Use: Learning branches and pull requests

Chat Link or Prompt/Response: See Appendix, Entry 3

AI-Suggested Content: How to create feature branches from dev, commit files, and open PRs into dev.

Validation: Followed our team's agreed workflow. Created feature/database-profiles and feature/database-resumes from dev and opened a PR for each into dev.

Decision: Accepted.

Reflection: Helped me understand the PR process and follow the team workflow.

Responsible Person: Yousef Yousef

=====
APPENDIX 3 - Git workflow for committing the database schema (full prompts and responses)
=====

--- Prompt 1 (2026-09-24) ---

we said whenever we want to push something, a new feature or whatever, we push from dev. So we we create a new branch from dev. We'll call it like you know feature slash. And then the feature, right? So how do I create a new branch off of dev? Put this feature and then eventually push it back to dev.

--- Response 1 ---

Yes, that makes sense. It's the standard feature branch workflow: dev → feature/database-profiles → (your work) → pull request → back into dev. Select dev first. This step matters, because the new branch copies whichever branch is selected.

--- Prompt 2 (2026-09-24) ---

Should I even commit it yet? Or do I continue building on it? And if I do commit it now, give me step by step what to do, branch, PR, all that.

--- Response 2 ---

Commit now, after a 2-minute test. The profiles table is a complete piece of work, and small PRs are better than one giant one. Create your branch from dev. Add the file and commit. Open the pull request. Closes #4. under Reviewers, add Catalin.

--- Prompt 3 (2026-09-24) ---

ok now back to committing the changes of file storage and resumes table. my profiles pr didnt get accepted yet

--- Response 3 ---

Since the profiles PR isn't merged, the resumes work goes on a new branch from dev, in a separate file. Name: supabase/resumes.sql. Depends on the profiles PR (resumes references profiles), merge that one first.

Entry number: 4

Date: 2026-09-26

AI tool/model: Claude (Anthropic), claude-opus-5-5

Task ID/Title: PR #46 review - Resume upload frontend (User Story #3, #19)

Purpose of AI Use: Code review and debugging of a teammate's pull request.

Chat Link or Prompt/Response: Appendix 4.

AI-Suggested Content: Found that the page showed "File uploaded successfully" without uploading anything to Supabase, and that the lowercase component name (uploadResumePage) breaks React's naming rule. Gave commands and test cases to verify the PR locally, and explained what ESLint checks.

Validation: Ran ESLint on the file, which returned 3 rules-of-hooks errors, confirming the naming issue. Tested the page in the browser (5 MB limit worked) and confirmed nothing reached Supabase Storage.

Decision: Accepted. I fixed the component name in PR #56. The upload was later connected to Supabase in PR #57, which I tested and merged.

Reflection: The dev server does not run lint or type checks, so the error was invisible in the browser. I now run lint and tsc before approving PRs.

Responsible Person: Yousef Yousef

=====

APPENDIX 4 - PR #46 review (full prompts and responses)

=====

--- Prompt 1 (2026-09-26, 21:28) ---

check the repo for this, and it tell me if the new PR is good and how to check in vs myself:

Created the resume upload Frontend- #46

#46

--- Response 1 ---

Verdict: good start, but it's frontend only. Nothing actually uploads to Supabase yet. It's fine to merge as a step, with a couple of small fixes.

What's good:

- New page at /upload-resume, one file, no conflicts with dev.
- Checks the file is a PDF and 5 MB or less, matching your bucket rules.
- Select, remove (X), and upload buttons with status messages. Clean and well commented.

Issues:

1. It says "File uploaded successfully" but uploads nothing. There's no Supabase call, so no file goes to the bucket and no row goes into the resumes table. That message is misleading until the backend is wired. It should say something like "Upload coming soon", or the PR should be clearly labeled frontend-only (the title does say "Frontend").

2. The component name is lowercase (uploadResumePage). React components should start with a capital letter (UploadResumePage). The lint rule for hooks will likely flag useState inside a lowercase function.
3. Minor: no login check on the page yet. The backend step will need the logged-in user's id for the <user id>/<file name> path anyway.

The missing part is your backend piece: upload the file to the resumes bucket at <user id>/<file name>, then insert a row into resumes. Good thing to pair on with Matthew next.

Check it yourself in VS Code (stop the app with Ctrl + C first). From careerconnect:

```
git fetch origin pull/46/head:pr-46
```

```
git switch pr-46
```

```
cd web
```

```
npx tsc --noEmit
```

```
npm run lint
```

```
npm run dev
```

- tsc should show no errors. lint will probably complain about the lowercase name.
- Open localhost:3000/upload-resume and try:
 - a PDF under 5 MB: accepted
 - a non-PDF (e.g. a .png): rejected
 - a PDF over 5 MB: rejected
 - the X button: removes the file
 - Upload with no file selected: error message
- Check Supabase Storage > resumes: it'll be empty, which confirms issue #1.

When done, cd .. then git switch dev.

Review comment (Submit review > Comment or Approve, your call):

"Validation + UI work well. Two things: rename the component to UploadResumePage (capital U, lint flags hooks in lowercase functions), and the "uploaded successfully" message shows even though nothing reaches Supabase yet. Happy to pair on wiring the upload to the resumes bucket + table next."

--- Prompt 2 (2026-09-26, 21:32) ---

C:\Users\naugh\Documents\careerconnect\web\src\app\upload-resume\page.tsx

6:53 error React Hook "useState" is called in function "uploadResumePage" that is neither a React function component nor a custom React Hook function. React component names must start with an uppercase letter. React Hook names must start with the word "use" react-hooks/rules-of-hooks

7:55 error React Hook "useState" is called in function "uploadResumePage" that is neither a React function component nor a custom React Hook function. React component names must start with an uppercase letter. React Hook names must start with the word "use" react-hooks/rules-of-hooks

8:32 error React Hook "useRef" is called in function "uploadResumePage" that is neither a React function component nor a custom React Hook function. React component names must start with an uppercase letter. React Hook names must start with the word "use" react-hooks/rules-of-hooks

3 problems (3 errors, 0 warnings)

--- Response 2 ---

That confirms issue #2: 3 lint errors, all from the lowercase component name. The fix is one character: uploadResumePage becomes UploadResumePage on line 5.

Next, test the page:

npm run dev

Open localhost:3000/upload-resume and try:

- a PDF under 5 MB: should be accepted
- a non-PDF: should be rejected
- a PDF over 5 MB: should be rejected
- the X button: removes the file
- Upload with no file: error message

Then check Supabase > Storage > resumes; it should be empty.

Then review #46 with Request changes, since lint errors should be fixed before merging:

"Validation + UI work well. Two things before merge:

1. npm run lint gives 3 errors: component needs a capital letter > UploadResumePage (line 5)
2. "File uploaded successfully" shows but nothing reaches Supabase yet, maybe change the message until the upload's wired

Happy to pair on connecting it to the resumes bucket + table next"

When done: Ctrl + C, then git switch dev.

--- Prompt 3 (2026-09-26, 21:36) ---

what is npm lint

--- Response 3 ---

`npm run lint` runs ESLint, a tool that reads your code without running it and flags mistakes and bad patterns. It's like a spell-checker for code.

It's different from the other checks you ran:

- `npx tsc --noEmit` (TypeScript) checks types, like "this prop doesn't exist on Button" (Ray's `asChild` bug).
- `npm run lint` (ESLint) checks rules and conventions, like "hooks must be called in components with capital-letter names" (Matthew's bug). The code can have correct types and still break these rules.
- `npm run dev` just runs the app. It doesn't do either check fully, which is why both bugs looked fine in the browser.

The rules come from `web/eslint.config.mjs` (Next.js's recommended set). Running lint and tsc before approving PRs is a good habit, and later you could have GitHub Actions run them automatically on every PR. That's the CI part of the course.